

Plytix API v1 and v2 Rate Limits

Documentation brief for RightChoice

This document consolidates the confirmed rate-limit behaviour of the Plytix PIM API v1 and v2. It covers the account-level limits, how requests are counted, the `rate_limit` access-token claim, the HTTP 429 response and the retry behaviour expected from integrations.

Rate limits

Plytix evaluates two rate-limit windows concurrently. A request must remain within both limits.

Account plan	Short-window limit	Hourly limit
Free	20 requests per 10 seconds	2,000 requests per hour
PRO and Standard+	50 requests per 10 seconds	5,000 requests per hour

The short window controls bursts. The hourly window controls sustained usage. An account can remain below the short-window limit and still reach the hourly limit.

Once a limit is reached, subsequent requests return HTTP 429 Too Many Requests until the exceeded window expires.

Account-level enforcement

Rate limits apply per account. All API keys and sessions belonging to the same account share the same request allowance.

Creating additional API keys does not increase the account's request capacity. Integrations that use several API keys, workers or services must coordinate their combined request volume against the same limits.

Request counting

Every request to a PIM API resource counts as one hit. This applies to read and write operations across API v1 and v2, including `GET`, `POST`, `PUT`, `PATCH` and `DELETE` requests.

Read and write operations use the same account-level rate-limit buckets. Requests made after a limit has been reached also count as API requests and receive a 429 response until the window expires.

The rate limit token claim

The account's rate-limit configuration is included in the access token under:

```
user_claims.account.rate_limit
```

For a PRO or Standard+ account, the claim can be represented as follows:

```
{
  "user_claims": {
    "account": {
      "rate_limit": [
        {
          "limit": 50,
          "window_size": 10
        }
      ]
    }
  }
}
```

```

    },
    {
      "limit": 5000,
      "window_size": 3600
    }
  ]
}
}
}

```

Field	Type	Unit	Meaning
rate_limit	Array	Not applicable	Rate-limit windows configured for the account
limit	Integer	Requests	Maximum number of requests allowed within the window
window_size	Integer	Seconds	Duration of the corresponding window

Each object in the array represents an independent limit. All configured windows are evaluated concurrently.

The claim describes the account's configuration. It does not show current consumption, remaining requests, the start or end of a window, or the time remaining before a reset.

HTTP 429 response

When an account exceeds one of its configured limits, the PIM API returns:

```
429 Too Many Requests
```

The response identifies the window that was exceeded.

Response body

```

{
  "message": "API rate limit exceeded",
  "limit": 5000,
  "window_size": 3600
}

```

Field	Type	Meaning
message	String	Description of the rate-limit error
limit	Integer	Request threshold of the exceeded window
window_size	Integer	Duration of the exceeded window in seconds

Response headers

```

Retry-After: 347
RateLimit-Limit: 5000
RateLimit-Window: 3600

```

Header	Format	Meaning
Retry-After	Integer in delta-seconds	Remaining TTL of the exceeded window
RateLimit-Limit	Integer	Request threshold of the exceeded window
RateLimit-Window	Integer in seconds	Full duration of the exceeded window

HTTP header names are case-insensitive.

Retry After behaviour

Plytix returns `Retry-After` as a numeric duration in seconds, not as an HTTP date.

For example:

```
Retry-After: 347
RateLimit-Window: 3600
```

This means that the exceeded window lasts 3,600 seconds and has 347 seconds remaining. The integration must wait at least 347 seconds before retrying.

`Retry-After` represents the remaining TTL of the active window. It is therefore not necessarily equal to the full `RateLimit-Window` value.

Response examples

Short-window limit

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 6
RateLimit-Limit: 50
RateLimit-Window: 10

{
  "message": "API rate limit exceeded",
  "limit": 50,
  "window_size": 10
}
```

The account exceeded 50 requests within 10 seconds. Six seconds remain before the window expires.

Hourly limit

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 1247
RateLimit-Limit: 5000
RateLimit-Window: 3600

{
  "message": "API rate limit exceeded",
  "limit": 5000,
  "window_size": 3600
}
```

The account exceeded 5,000 requests within one hour. The integration must wait at least 1,247 seconds before retrying.

Recommended retry behaviour

When an integration receives a 429 response, it should:

1. Read the `Retry-After` header.
2. Interpret the value as seconds.
3. Pause account requests for at least that duration.
4. Add a small random delay before resuming to prevent several workers from retrying simultaneously.
5. Retry the request after the waiting period.
6. Set a maximum number of retries and record persistent failures.

Example JavaScript implementation:

```
const sleep = (milliseconds) =>
  new Promise((resolve) => setTimeout(resolve, milliseconds));

async function fetchWithRateLimitRetry(url, options = {}, maxRetries = 5) {
  for (let attempt = 0; attempt <= maxRetries; attempt += 1) {
    const response = await fetch(url, options);

    if (response.status !== 429 || attempt === maxRetries) {
      return response;
    }

    const retryAfter = Number.parseInt(
      response.headers.get("Retry-After"),
      10
    );

    const jitterMilliseconds = Math.floor(Math.random() * 250);
    await sleep(retryAfter * 1000 + jitterMilliseconds);
  }
}
```

Integration recommendations

- Apply client-side throttling for both configured windows.
- Coordinate request counts across every API key and worker used by the account.
- Use a central request queue when an integration runs across several processes or services.
- Avoid sending large bursts of requests.
- Use pagination, search and batch operations to reduce the total number of requests.
- Retrieve only the fields required by the integration.
- Cache data that does not need to be retrieved repeatedly.
- Log the returned `limit`, `window_size` and `Retry-After` values when a 429 occurs.
- Monitor repeated 429 responses as an integration health indicator.

Authentication service

Access tokens are obtained from:

```
POST https://auth.plytix.com/auth/api/get-token
```

The authentication service has its own service-level rate limiting. Its allowance is separate from the account-level PIM API limits described in this document.

Access tokens remain valid for 15 minutes. Integrations should reuse the same token during that period instead of requesting a new token before every PIM API call.

Summary

- Rate limits apply per account and are shared by all API keys and sessions.
- Free accounts allow 20 requests per 10 seconds and 2,000 requests per hour.
- PRO and Standard+ accounts allow 50 requests per 10 seconds and 5,000 requests per hour.
- The short and hourly windows are evaluated concurrently.
- Read and write requests use the same account allowance.
- The access token's `rate_limit` claim describes the configured limits rather than current usage.
- A rate-limited request returns HTTP 429 and identifies the exceeded window.
- `Retry-After` is expressed in seconds and contains the remaining TTL of that window.
- Integrations must coordinate throttling and retries across the complete account.